In [1]:
```
# This Python 3 environment comes with many helpful analytics libraries i
# It is defined by the kaggle/python Docker image: https://github.com/kag
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt
# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) wil

import os
# for dirname, _, filenames in os.walk('/kaggle/input'):
#     for filename in filenames:
#         pass
        # print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) th
# You can also write temporary files to /kaggle/temp/, but they won't be
```

In [2]:
```
import json
import hashlib
from PIL import Image
import torch
from torchvision import transforms
import torch.nn.functional as F
from torchvision.utils import save_image
```

In [3]:
```
import os
import json
import hashlib
import numpy as np
from PIL import Image
import torch
import torch.nn.functional as F
from torchvision.utils import save_image
import random

def load_image_to_tensor(img_path, device):
    """Load image and return float tensor in range [0,1] on specified dev
    img = Image.open(img_path).convert("RGB")
    np_img = np.array(img, dtype=np.uint8)        # (H, W, C)
    tensor_img = torch.from_numpy(np_img).permute(2,0,1).float() / 255.0
    return tensor_img.to(device), img.size  # (C,H,W), (W,H)

def preprocess_multiple_folders_gpu(
    input_folders,
    output_folder="processed_images",
    target_size=128,
    padding_color=(0, 0, 0),
    metadata_file="image_metadata.json",
    batch_size=32,
    max_files_per_folder=None,
    shuffle_files=True,
    save_masks=True,
    train_split_ratio=0.8,     # proportion of training data
    seed=42
):
    # Set seeds for reproducibility
```

```
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

    os.makedirs(output_folder, exist_ok=True)
    if save_masks:
        mask_dir = os.path.join(output_folder, "masks")
        os.makedirs(mask_dir, exist_ok=True)

    valid_exts = ('.jpg', '.jpeg', '.png')

    all_image_paths = []
    for folder in input_folders:
        folder_image_paths = []
        for root, _, files in os.walk(folder):
            for fname in files:
                if fname.lower().endswith(valid_exts):
                    folder_image_paths.append(os.path.join(root, fname))

        if shuffle_files:
            random.shuffle(folder_image_paths)

        if max_files_per_folder is not None:
            folder_image_paths = folder_image_paths[:max_files_per_folder

        all_image_paths.extend(folder_image_paths)
        print(f"{folder}: selected {len(folder_image_paths)} files")

    print(f"Total images to process: {len(all_image_paths)}")

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    metadata = {}
    saved_filenames = []

    for start in range(0, len(all_image_paths), batch_size):
        batch_paths = all_image_paths[start:start+batch_size]
        batch_tensors = []
        batch_info = []

        for img_path in batch_paths:
            try:
                tensor_img, (original_w, original_h) = load_image_to_tens
                aspect_ratio = original_w / original_h

                scale = target_size / max(original_w, original_h)
                new_w = max(1, int(original_w * scale))
                new_h = max(1, int(original_h * scale))

                resized_img = F.interpolate(
                    tensor_img.unsqueeze(0),
                    size=(new_h, new_w),
                    mode='bilinear',
                    align_corners=False
                ).squeeze(0)

                new_img = torch.ones((3, target_size, target_size), devic
                for i in range(3):
                    new_img[i] *= padding_color[i] / 255.0
```

```
            paste_x = (target_size - new_w) // 2
            paste_y = (target_size - new_h) // 2
            new_img[:, paste_y:paste_y+new_h, paste_x:paste_x+new_w]
            batch_tensors.append(new_img)

            base_name = os.path.splitext(os.path.basename(img_path))[
            hash_suffix = hashlib.md5(img_path.encode('utf-8')).hexdi
            new_filename = f"{base_name}_{hash_suffix}.jpg"

            saved_filenames.append(new_filename)

            mask_path = None
            if save_masks:
                mask = torch.zeros((1, target_size, target_size), dty
                mask[:, paste_y:paste_y+new_h, paste_x:paste_x+new_w]
                mask_path = os.path.join(mask_dir, new_filename.repla
                torch.save(mask.cpu(), mask_path)

            batch_info.append((
                new_filename,
                {
                    "original_path": img_path,
                    "original_size": (original_w, original_h),
                    "resized_size": (new_w, new_h),
                    "scale_factor": scale,
                    "padding_offsets": (paste_x, paste_y),
                    "aspect_ratio": aspect_ratio,
                    "mask_path": mask_path
                }
            ))

        except Exception as e:
            print(f"Failed to process {img_path}: {e}")

    # Save images and metadata on CPU
    for tensor_img, (filename, meta) in zip(batch_tensors, batch_info
        save_image(tensor_img.cpu(), os.path.join(output_folder, file
        metadata[filename] = meta

    print(f"Processed batch {start//batch_size + 1} / {(len(all_image

# Save metadata JSON
meta_path = os.path.join(output_folder, metadata_file)
with open(meta_path, "w") as f:
    json.dump(metadata, f, indent=4)
print(f"Metadata saved to {meta_path}")

# Shuffle and split dataset filenames into train and val
random.shuffle(saved_filenames)
split_index = int(len(saved_filenames) * train_split_ratio)
train_files = saved_filenames[:split_index]
val_files = saved_filenames[split_index:]

train_split_path = os.path.join(output_folder, "train_split.json")
val_split_path = os.path.join(output_folder, "val_split.json")

with open(train_split_path, "w") as f:
    json.dump(train_files, f)
with open(val_split_path, "w") as f:
```

```
        json.dump(val_files, f)

    print(f"Train/Val split saved: {len(train_files)} train, {len(val_fil
    print(f"All images saved to {output_folder}")
    if save_masks:
        print(f"Masks saved to {mask_dir}")
```

In [4]:
```
TARGET_SIZE = 256
```

In [5]:
```
preprocess_multiple_folders_gpu(
    ["/kaggle/input/wikiart/Symbolism",
     # "/kaggle/input/wikiart/Art_Nouveau_Modern",
     # "/kaggle/input/wikiart/Ukiyo_e"
    ],
    output_folder="processed_images",
    target_size=TARGET_SIZE,
    padding_color=(0, 0, 0),
    metadata_file="image_metadata.json",
    batch_size=32,
    max_files_per_folder=None,
    shuffle_files=True      ,
    train_split_ratio=0.5,
    seed = 42
)
```

```
/kaggle/input/wikiart/Symbolism: selected 4528 files
Total images to process: 4528
Processed batch 1 / 142
Processed batch 2 / 142
Processed batch 3 / 142
Processed batch 4 / 142
Processed batch 5 / 142
Processed batch 6 / 142
Processed batch 7 / 142
Processed batch 8 / 142
Processed batch 9 / 142
Processed batch 10 / 142
Processed batch 11 / 142
Processed batch 12 / 142
Processed batch 13 / 142
Processed batch 14 / 142
Processed batch 15 / 142
Processed batch 16 / 142
Processed batch 17 / 142
Processed batch 18 / 142
Processed batch 19 / 142
Processed batch 20 / 142
Processed batch 21 / 142
Processed batch 22 / 142
Processed batch 23 / 142
Processed batch 24 / 142
Processed batch 25 / 142
Processed batch 26 / 142
Processed batch 27 / 142
Processed batch 28 / 142
Processed batch 29 / 142
Processed batch 30 / 142
Processed batch 31 / 142
Processed batch 32 / 142
Processed batch 33 / 142
Processed batch 34 / 142
Processed batch 35 / 142
Processed batch 36 / 142
Processed batch 37 / 142
Processed batch 38 / 142
Processed batch 39 / 142
Processed batch 40 / 142
Processed batch 41 / 142
Processed batch 42 / 142
Processed batch 43 / 142
Processed batch 44 / 142
Processed batch 45 / 142
Processed batch 46 / 142
Processed batch 47 / 142
Processed batch 48 / 142
Processed batch 49 / 142
Processed batch 50 / 142
Processed batch 51 / 142
Processed batch 52 / 142
Processed batch 53 / 142
Processed batch 54 / 142
Processed batch 55 / 142
Processed batch 56 / 142
Processed batch 57 / 142
Processed batch 58 / 142
```

```
Processed batch 59 / 142
Processed batch 60 / 142
Processed batch 61 / 142
Processed batch 62 / 142
Processed batch 63 / 142
Processed batch 64 / 142
Processed batch 65 / 142
Processed batch 66 / 142
Processed batch 67 / 142
Processed batch 68 / 142
Processed batch 69 / 142
Processed batch 70 / 142
Processed batch 71 / 142
Processed batch 72 / 142
Processed batch 73 / 142
Processed batch 74 / 142
Processed batch 75 / 142
Processed batch 76 / 142
Processed batch 77 / 142
Processed batch 78 / 142
Processed batch 79 / 142
Processed batch 80 / 142
Processed batch 81 / 142
Processed batch 82 / 142
Processed batch 83 / 142
Processed batch 84 / 142
Processed batch 85 / 142
Processed batch 86 / 142
Processed batch 87 / 142
Processed batch 88 / 142
Processed batch 89 / 142
Processed batch 90 / 142
Processed batch 91 / 142
Processed batch 92 / 142
Processed batch 93 / 142
Processed batch 94 / 142
Processed batch 95 / 142
Processed batch 96 / 142
Processed batch 97 / 142
Processed batch 98 / 142
Processed batch 99 / 142
Processed batch 100 / 142
Processed batch 101 / 142
Processed batch 102 / 142
Processed batch 103 / 142
Processed batch 104 / 142
Processed batch 105 / 142
Processed batch 106 / 142
Processed batch 107 / 142
Processed batch 108 / 142
Processed batch 109 / 142
Processed batch 110 / 142
Processed batch 111 / 142
Processed batch 112 / 142
Processed batch 113 / 142
Processed batch 114 / 142
Processed batch 115 / 142
Processed batch 116 / 142
Processed batch 117 / 142
Processed batch 118 / 142
```

```
Processed batch 119 / 142
Processed batch 120 / 142
Processed batch 121 / 142
Processed batch 122 / 142
Processed batch 123 / 142
Processed batch 124 / 142
Processed batch 125 / 142
Processed batch 126 / 142
Processed batch 127 / 142
Processed batch 128 / 142
Processed batch 129 / 142
Processed batch 130 / 142
Processed batch 131 / 142
Processed batch 132 / 142
Processed batch 133 / 142
Processed batch 134 / 142
Processed batch 135 / 142
Processed batch 136 / 142
Processed batch 137 / 142
Processed batch 138 / 142
Processed batch 139 / 142
Processed batch 140 / 142
Processed batch 141 / 142
Processed batch 142 / 142
Metadata saved to processed_images/image_metadata.json
Train/Val split saved: 2264 train, 2264 val
All images saved to processed_images
Masks saved to processed_images/masks
```

In [6]:

```python
from torch.utils.data import Dataset
from PIL import Image
import torch
import json
import os

class LazyAutoencoderDataset(Dataset):
    def __init__(
        self,
        image_dir,
        file_list_json,
        metadata_json=None,          # ✅ Optional: path to metadata.json
        input_transform=None,
        target_transform=None,
        load_masks=False             # ✅ Load precomputed masks if availa
    ):
        self.image_dir = image_dir
        with open(file_list_json, "r") as f:
            self.file_list = json.load(f)

        self.input_transform = input_transform
        self.target_transform = target_transform
        self.load_masks = load_masks
        self.metadata = None

        if metadata_json and os.path.exists(metadata_json):
            with open(metadata_json, "r") as f:
                self.metadata = json.load(f)

    def __len__(self):
        return len(self.file_list)
```

```
    def __getitem__(self, idx):
        filename = self.file_list[idx]
        img_path = os.path.join(self.image_dir, filename)
        img = Image.open(img_path).convert("RGB")

        # Input: possibly augmented
        x = self.input_transform(img) if self.input_transform else img
        # Target: clean, normalized
        y = self.target_transform(img) if self.target_transform else x

        mask = None
        aspect_ratio = None
        if self.metadata:
            meta = self.metadata.get(filename, {})
            aspect_ratio = meta.get("aspect_ratio", None)

            if self.load_masks and "mask_path" in meta and meta["mask_pat
                mask = torch.load(meta["mask_path"])  # (1,H,W) tensor

        return {
            "input": x,
            "target": y,
            "filename": filename,
            "mask": mask,                    # ✅ optional (None if not lo
            "aspect_ratio": aspect_ratio     # ✅ optional
        }
```

In [7]:
```
from torchvision.transforms import functional as TF

class RandomMask:
    def __init__(self, mask_ratio=0.3):
        self.mask_ratio = mask_ratio
    def __call__(self, img_tensor):
        c, h, w = img_tensor.shape
        mask = torch.ones((h, w))
        num_pixels = int(h*w*self.mask_ratio)
        idx = torch.randperm(h*w)[:num_pixels]
        mask.view(-1)[idx] = 0
        mask = mask.unsqueeze(0).expand(c, -1, -1)
        return img_tensor * mask
```

In [8]:
```
class GaussianNoise:
    def __init__(self, sigma=0.1):
        self.sigma = sigma
    def __call__(self, tensor):
        return tensor + torch.randn_like(tensor)*self.sigma
```

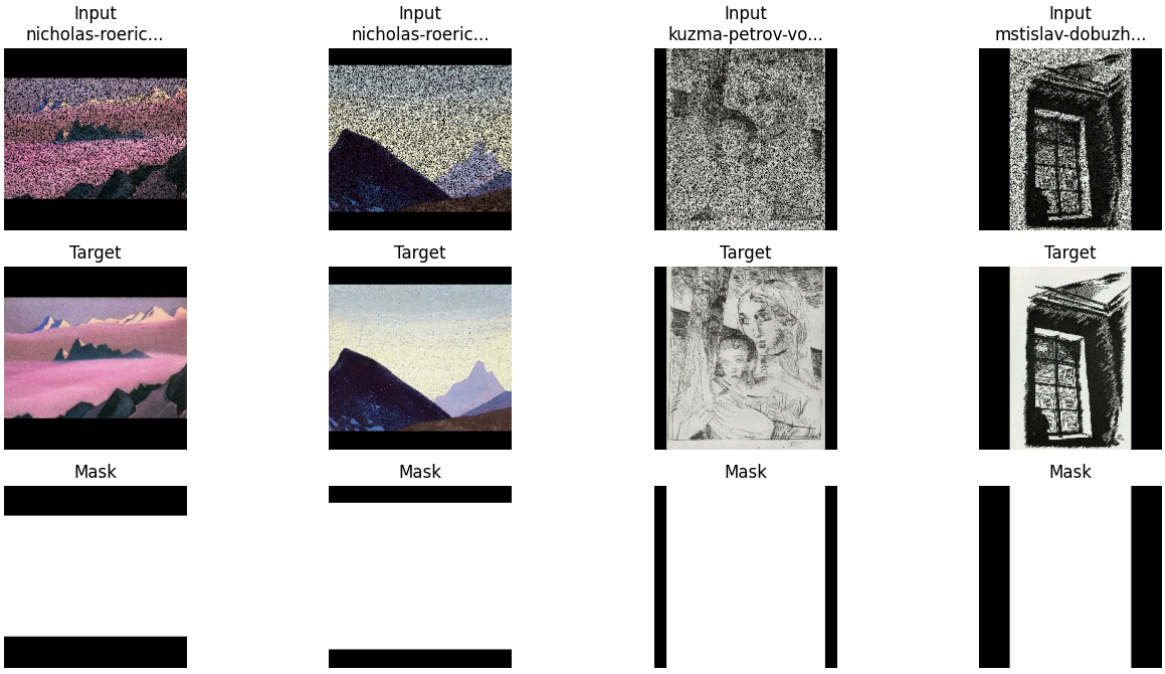
In [9]:
```
input_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    RandomMask(mask_ratio=0.3),  # 30% of pixels masked,
    GaussianNoise(0.0005)
])

target_transform = transforms.Compose([
    transforms.ToTensor(),
])
```

In [10]:
```
train_dataset = LazyAutoencoderDataset(
    image_dir="processed_images",
    file_list_json="processed_images/train_split.json",  # ✅ list
    metadata_json="processed_images/image_metadata.json", # ✅ optional
    input_transform=input_transform,
    target_transform=target_transform,
    load_masks=True
)

val_dataset = LazyAutoencoderDataset(
    image_dir="processed_images",
    file_list_json="processed_images/val_split.json",    # ✅ list
    metadata_json="processed_images/image_metadata.json",# ✅ optional
    input_transform=input_transform,
    target_transform=target_transform,
    load_masks=True
)
```

In [11]:
```
import matplotlib.pyplot as plt

def visualize_samples(dataset, indices=None, n=8, show_mask=False):
    """
    Visualize samples from LazyAutoencoderDataset that returns dicts.

    Args:
        dataset: LazyAutoencoderDataset instance
        indices: Optional list of indices to visualize
        n: Number of samples if indices is None
        show_mask: Whether to visualize mask if available
    """
    if indices is None:
        indices = list(range(min(n, len(dataset))))  # first n by default

    plt.figure(figsize=(4*len(indices), 8 if show_mask else 6))

    for i, idx in enumerate(indices):
        sample = dataset[idx]
        x, y = sample["input"], sample["target"]
        x_np = x.permute(1, 2, 0).numpy()
        y_np = y.permute(1, 2, 0).numpy()

        # Input image
        plt.subplot(3 if show_mask else 2, len(indices), i + 1)
        plt.imshow(x_np)
        plt.axis("off")
        plt.title(f"Input\n{sample['filename'][:15]}...")

        # Target image
        plt.subplot(3 if show_mask else 2, len(indices), i + 1 + len(indi
        plt.imshow(y_np)
        plt.axis("off")
        plt.title("Target")

        # Optional mask
        if show_mask and sample["mask"] is not None:
            mask_np = sample["mask"].squeeze().numpy()
            plt.subplot(3, len(indices), i + 1 + 2*len(indices))
            plt.imshow(mask_np, cmap="gray")
            plt.axis("off")
```

```
            plt.title("Mask")

    plt.show()
```

In [12]:
```
visualize_samples(train_dataset, indices=[0, 1, 2, 3], show_mask=True)
```



In [13]:
```
import math
```

In [14]:
```
import torch.nn as nn
# -------------------------
# Model Components
# -------------------------

# 1. Patch Encoder
class PatchEncoder(nn.Module):
    """
    Encodes a single image patch into an embedding vector.
    Uses a small ConvNet + global pooling + linear projection.
    """
    def __init__(self, in_channels=3, embed_dim=128):
        super().__init__()
        # Convolutional backbone downsampling by factor 8
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, 32, kernel_size=3, stride=2, padding=1
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(64,128, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d(1)  # 1×1 spatial
        )
        # Project feature vector to embed_dim
        self.proj = nn.Linear(128, embed_dim)

    def forward(self, x):
        B = x.size(0)
        # Extract features and flatten
        f = self.net(x).view(B, 128)
        # Linear projection to embedding
```

```
        return self.proj(f)

# 2. Aspect Ratio Embedding
class AspectEmbedding(nn.Module):
    """
    Maps a scalar aspect ratio (W/H) into a learned embedding.
    Conditions the model on overall image shape.
    """
    def __init__(self, embed_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, embed_dim),
            nn.ReLU(),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, ratio):
        # ratio: tensor of shape (B,1)
        return self.net(ratio)

# 3. Patch Decoder
class PatchDecoder(nn.Module):
    """
    Decodes a latent vector into an image patch of exact size.
    Also predicts a binary mask for content vs padding.
    """
    def __init__(self, out_channels=3, patch_size=(32,32), latent_dim=128
        super().__init__()
        H, W = patch_size
        # Compute minimal starting grid dims (ceil division by 8)
        self.h8 = math.ceil(H/8)
        self.w8 = math.ceil(W/8)
        self.patch_size = (H, W)
        # Project latent to feature map
        self.fc = nn.Linear(latent_dim, 128 * self.h8 * self.w8)
        # Three upsampling blocks (×2 each) to reach at least 8× size
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.ReLU(),  # ×2
            nn.ConvTranspose2d(64, 32, 4, 2, 1),  nn.ReLU(),  # ×2
            nn.ConvTranspose2d(32, out_channels, 4, 2, 1), nn.Sigmoid()
        )
        # 1×1 conv to predict content mask
        self.mask_conv = nn.Conv2d(out_channels, 1, kernel_size=1)

    def forward(self, z):
        B = z.size(0)
        # Project and reshape to (B,128,h8,w8)
        x = self.fc(z).view(B, 128, self.h8, self.w8)
        # Upsample
        x = self.deconv(x)
        # Crop or pad to exact (H, W)
        H, W = self.patch_size
        ch, cw = x.size(2), x.size(3)
        if ch >= H and cw >= W:
            sh, sw = (ch-H)//2, (cw-W)//2
            x = x[:, :, sh:sh+H, sw:sw+W]
        else:
            ph, pw = max(0, H-ch), max(0, W-cw)
            x = F.pad(x, (pw//2, pw-pw//2, ph//2, ph-ph//2))
        # Predict mask
```

```
        mask_logits = self.mask_conv(x)
        mask_pred = torch.sigmoid(mask_logits)
        return x, mask_pred

# 4. Transformer Block
class TransformerBlock(nn.Module):
    """
    Single layer of a Transformer encoder with optional mask:
    - Multi-head self-attention (with attn_mask)
    - Residual + LayerNorm
    - Feed-forward network
    """
    def __init__(self, embed_dim=128, num_heads=4, ff_dim=256, dropout=0.
        super().__init__()
        self.attn = nn.MultiheadAttention(embed_dim, num_heads, batch_fir
        self.norm1 = nn.LayerNorm(embed_dim)
        self.ff = nn.Sequential(
            nn.Linear(embed_dim, ff_dim),
            nn.ReLU(),
            nn.Linear(ff_dim, embed_dim)
        )
        self.norm2 = nn.LayerNorm(embed_dim)
        self.drop = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        """
        x: (B, seq_len, D)
        mask: (seq_len, seq_len) boolean tensor where True = keep, False
        """
        attn_mask = None
        if mask is not None:
            # nn.MultiheadAttention expects float mask: (seq_len, seq_len
            # True -> 0.0, False -> -inf
            attn_mask = (~mask).float() * -1e9

        # Self-attention with optional causal mask
        attn_out, _ = self.attn(x, x, x, attn_mask=attn_mask)
        x = self.norm1(x + self.drop(attn_out))

        # Feed-forward pass
        ff_out = self.ff(x)
        x = self.norm2(x + self.drop(ff_out))
        return x

# -------------------------
# 5. Context-Aware Patch Generator
# -------------------------
class ContextPatchGenerator(nn.Module):
    def __init__(
        self,
        patch_size=(32,32),
        embed_dim=128,
        num_patches=12,
        num_heads=4,
        ff_dim=256,
        num_layers=2,
        in_channels=3,
        out_channels=3
    ):
        super().__init__()
```

```
        self.encoder = PatchEncoder(in_channels, embed_dim)
        self.aspect_emb = AspectEmbedding(embed_dim)
        self.context_table = nn.Parameter(torch.zeros(num_patches, embed_
        self.transformers = nn.ModuleList([
            TransformerBlock(embed_dim, num_heads, ff_dim, dropout=0.2)
            for _ in range(num_layers)
        ])
        self.to_latent = nn.Linear(embed_dim, embed_dim)
        self.decoder = PatchDecoder(out_channels, patch_size, embed_dim)

    def forward(self, inputs):
        """
        inputs: tuple of (patches_list, hw_tensor)
        patches_list: list of length num_patches of (B,C,H,W) tensors
        hw_tensor: (B,2) tensor of raw H,W
        """
        patches, hw = inputs
        patch_tensor = torch.stack(patches, dim=1)   # (B,P,C,H,W)
        device = patch_tensor.device
        B, P, C, H, W = patch_tensor.shape
        D = self.context_table.size(-1)

        # Move hw to correct device
        hw = hw.to(device)

        # Encode all patches at once: (B,P,D)
        patch_embs = []
        for i in range(P):
            patch_embs.append(self.encoder(patch_tensor[:, i]))
        patch_embs = torch.stack(patch_embs, dim=1)  # (B,P,D)

        # Aspect embedding
        ratio = (hw[:,1:2] / hw[:,0:1]).to(device)          # (B,1)
        aspect = self.aspect_emb(ratio).unsqueeze(1)        # (B,1,D)

        # Prepare sequence: patches + aspect token
        seq = torch.cat([patch_embs, aspect], dim=1)        # (B,P+1,D)

        # Create causal mask: (P+1,P+1)
        # Last token (aspect) attends to all
        causal_mask = torch.tril(torch.ones(P+1, P+1, device=device)).boo
        causal_mask[-1, :] = 1  # aspect token attends to all

        # Pass through transformers
        for layer in self.transformers:
            seq = layer(seq, mask=causal_mask)  # transformer must suppor

        # Project patch slots to latent
        z = self.to_latent(seq[:, :P])   # (B,P,D)

        # Decode all patches in parallel
        outputs, masks = [], []
        for i in range(P):
            img_out, mask_out = self.decoder(z[:, i])
            outputs.append(img_out)
            masks.append(mask_out)

        return outputs, masks
```

In [15]:
```
from torch.utils.data import DataLoader
```

In [16]:
```
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True, nu
val_loader   = DataLoader(val_dataset, batch_size=128, shuffle=False, num
```

In [17]:
```
# Instantiate model and optimizer
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
patch_size=(16,16)
model = ContextPatchGenerator(
    patch_size=patch_size, embed_dim=64, num_patches=int(TARGET_SIZE/patc
    num_heads=8, ff_dim=128, num_layers=4,
    in_channels=3, out_channels=3
).to(device)

# Distribute model across all GPUs if available
if torch.cuda.device_count() > 1:
    print(f"Using {torch.cuda.device_count()} GPUs with DataParallel")
    model = nn.DataParallel(model)

model = model.to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
```

```
Using 2 GPUs with DataParallel
```

In [18]:
```
import csv
from IPython.display import clear_output
```

In [19]:
```
def train_model(model, train_loader, val_loader, device, epochs=1000, lr=
                patience=100, log_path="training_log.csv",
                checkpoint_path="best_model.pt", kl_warmup_epochs=10,
                mask_weight_init=2.0, mask_weight_final=1.0, mask_anneal_
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    ph, pw = patch_size
    best_val_loss = float("inf")
    patience_counter = 0

    with open(log_path, "w", newline="") as f:
        logger = csv.writer(f)
        logger.writerow([
            "epoch", "train_loss", "val_loss",
            "train_recon", "train_mask", "train_kl",
            "val_recon", "val_mask", "val_kl", "kl_weight", "mask_weight"
        ])

        for epoch in range(epochs):
            model.train()
            train_loss = train_recon_loss = train_mask_loss = train_kl_lo

            # KL warmup
            kl_weight = min(1.0, epoch / float(kl_warmup_epochs))

            # Mask loss annealing (linear decay from init to final)
            if epoch < mask_anneal_epochs:
                mask_weight = mask_weight_init - (mask_weight_init - mask
            else:
                mask_weight = mask_weight_final

            for batch in train_loader:
                x, y, mask = batch["input"].to(device), batch["target"].t
```

```
            B, C, H, W = x.shape
            hw = torch.tensor([[H, W]]*B, dtype=torch.float32, device

            # Split into patches
            patches = [x[:, :, i:i+ph, j:j+pw]
                       for i in range(0, H, ph)
                       for j in range(0, W, pw)]

            optimizer.zero_grad()
            recon_patches, pred_masks = model((patches, hw))  # tuple

            # Reassemble patches
            recon = torch.zeros_like(x)
            pmask = torch.zeros_like(mask)
            idx = 0
            for i in range(0, H, ph):
                for j in range(0, W, pw):
                    recon[:, :, i:i+ph, j:j+pw] = recon_patches[idx]
                    pmask[:, :, i:i+ph, j:j+pw] = pred_masks[idx]
                    idx += 1

            # Losses
            loss_recon = ((recon - y)**2 * mask).sum() / mask.sum()
            loss_mask = F.binary_cross_entropy(pmask, mask)
            kl_loss = torch.mean(torch.square(model.module.context_ta
                      if isinstance(model, nn.DataParallel) else \
                      torch.mean(torch.square(model.context_table))

            # ✅ Combine with weights
            loss = 10*loss_recon + mask_weight * loss_mask + kl_weigh
            loss.backward()
            optimizer.step()

            # Track losses
            train_loss += loss.item()
            train_recon_loss += loss_recon.item()
            train_mask_loss += loss_mask.item()
            train_kl_loss += kl_loss.item()

        # Epoch stats
        avg_train_loss = train_loss / len(train_loader)
        avg_train_recon = train_recon_loss / len(train_loader)
        avg_train_mask = train_mask_loss / len(train_loader)
        avg_train_kl = train_kl_loss / len(train_loader)

        # Validation
        model.eval()
        val_loss = val_recon_loss = val_mask_loss = val_kl_loss = 0.0
        with torch.no_grad():
            for batch in val_loader:
                x, y, mask = batch["input"].to(device), batch["target
                B, C, H, W = x.shape
                hw = torch.tensor([[H, W]]*B, dtype=torch.float32, de
                patches = [x[:, :, i:i+ph, j:j+pw]
                           for i in range(0, H, ph)
                           for j in range(0, W, pw)]

                recon_patches, pred_masks = model((patches, hw))
                recon = torch.zeros_like(x)
                pmask = torch.zeros_like(mask)
```

```
                    idx = 0
                    for i in range(0, H, ph):
                        for j in range(0, W, pw):
                            recon[:, :, i:i+ph, j:j+pw] = recon_patches[i
                            pmask[:, :, i:i+ph, j:j+pw] = pred_masks[idx]
                            idx += 1

                    loss_recon = ((recon - y)**2 * mask).sum() / mask.sum
                    loss_mask = F.binary_cross_entropy(pmask, mask)
                    kl_loss = torch.mean(torch.square(model.module.contex
                              if isinstance(model, nn.DataParallel) else
                              torch.mean(torch.square(model.context_table
                    batch_loss = 10*loss_recon + mask_weight * loss_mask

                    val_loss += batch_loss.item()
                    val_recon_loss += loss_recon.item()
                    val_mask_loss += loss_mask.item()
                    val_kl_loss += kl_loss.item()

            avg_val_loss = val_loss / len(val_loader)
            avg_val_recon = val_recon_loss / len(val_loader)
            avg_val_mask = val_mask_loss / len(val_loader)
            avg_val_kl = val_kl_loss / len(val_loader)

            logger.writerow([
                epoch+1, avg_train_loss, avg_val_loss,
                avg_train_recon, avg_train_mask, avg_train_kl,
                avg_val_recon, avg_val_mask, avg_val_kl, kl_weight, mask_
            ])
            print(f"Epoch {epoch+1}: "
                  f"Train={avg_train_loss:.4f} (Recon={avg_train_recon:.4
                  f"| Val={avg_val_loss:.4f} (Recon={avg_val_recon:.4f},
                  f"| KLw={kl_weight:.2f} | Mw={mask_weight:.2f}")
            clear_output(wait=True)

            # Early stopping
            if avg_val_loss < best_val_loss:
                best_val_loss = avg_val_loss
                torch.save(model.state_dict(), checkpoint_path)
                patience_counter = 0
            else:
                patience_counter += 1
                if patience_counter >= patience:
                    print("Early stopping triggered")
                    break
```

In [20]:

```
train_model(
    model,
    train_loader,
    val_loader,
    device,
    epochs=5000,
    lr=1e-3,
    patience=50,
    log_path="training_log.csv",
    checkpoint_path="best_model.pt",
    kl_warmup_epochs=100,        # long warmup for KL
    mask_weight_init=2.0,        # start with heavier mask loss
    mask_weight_final=0.5,       # gradually reduce to 1.0
    mask_anneal_epochs=100       # over first 200 epochs
```

```
)

# Clear output of the current cell
# clear_output(wait=True)
```

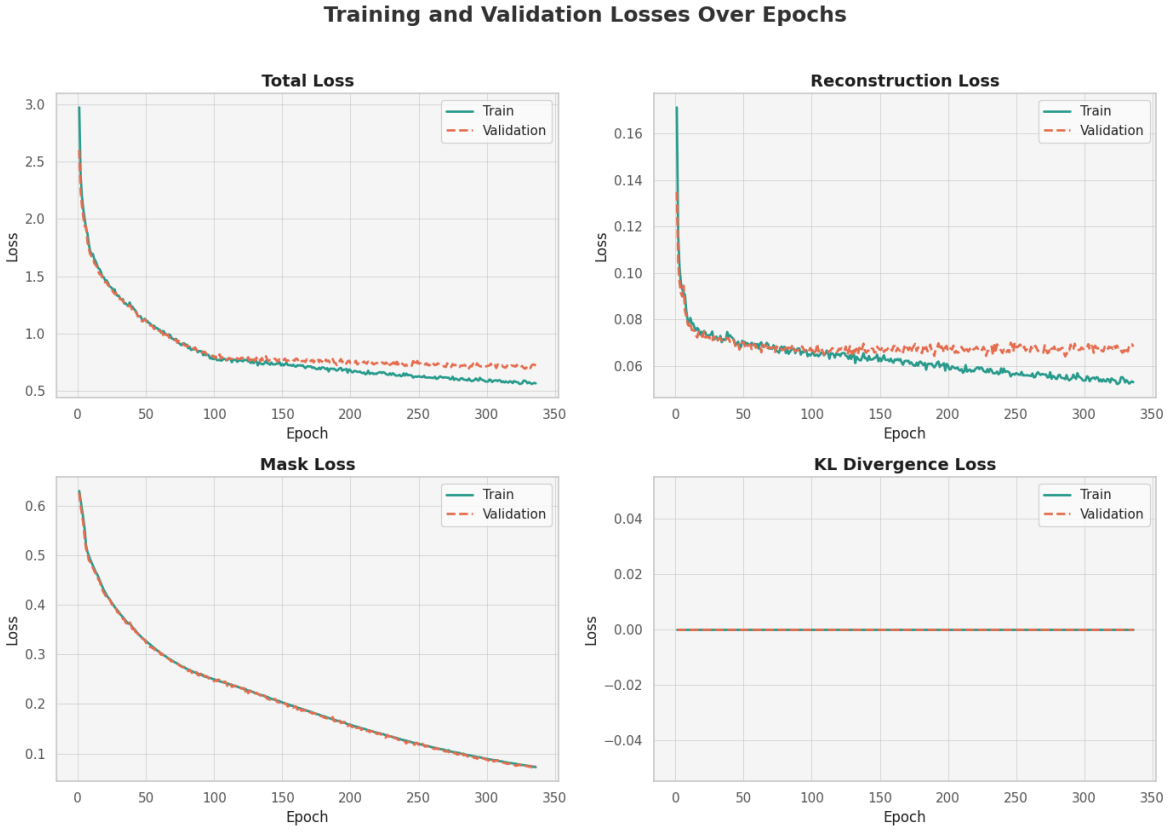
```
Early stopping triggered
```

In [21]:

```
import matplotlib.pyplot as plt
import seaborn as sns
import csv

def plot_training_log(log_path="training_log.csv"):
    epochs, train_loss, val_loss = [], [], []
    train_recon, train_mask, train_kl = [], [], []
    val_recon, val_mask, val_kl = [], [], []

    # Read CSV data
    with open(log_path, "r") as f:
        reader = csv.DictReader(f)
        for row in reader:
            epochs.append(int(row["epoch"]))
            train_loss.append(float(row["train_loss"]))
            val_loss.append(float(row["val_loss"]))
            train_recon.append(float(row["train_recon"]))
            train_mask.append(float(row["train_mask"]))
            train_kl.append(float(row["train_kl"]))
            val_recon.append(float(row["val_recon"]))
            val_mask.append(float(row["val_mask"]))
            val_kl.append(float(row["val_kl"]))

    # Use seaborn style with white grid background
    sns.set_theme(style="whitegrid")

    fig, axs = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle("Training and Validation Losses Over Epochs", fontsize=1

    # Custom colors for clarity
    train_color = "#2A9D8F"       # teal green
    val_color = "#E76F51"         # warm red-orange
    grid_color = '#cccccc'        # light gray grid

    def plot_loss(ax, epochs, train_data, val_data, title, ylabel):
        ax.set_facecolor('#f9f9f9')  # very light gray background for eac
        ax.plot(epochs, train_data, label="Train", color=train_color, lin
        ax.plot(epochs, val_data, label="Validation", color=val_color, li
        ax.set_title(title, fontsize=14, weight='semibold', color='#22222
        ax.set_xlabel("Epoch", fontsize=12, color='#222222')
        ax.set_ylabel(ylabel, fontsize=12, color='#222222')
        ax.legend(fontsize=11)
        ax.grid(True, linestyle='-', linewidth=0.7, color=grid_color, alp
        ax.tick_params(axis='x', colors='#555555')
        ax.tick_params(axis='y', colors='#555555')

    # Total Loss
    plot_loss(axs[0, 0], epochs, train_loss, val_loss, "Total Loss", "Los

    # Reconstruction Loss
    plot_loss(axs[0, 1], epochs, train_recon, val_recon, "Reconstruction

    # Mask Loss
```

```
    plot_loss(axs[1, 0], epochs, train_mask, val_mask, "Mask Loss", "Loss

    # KL Loss
    plot_loss(axs[1, 1], epochs, train_kl, val_kl, "KL Divergence Loss",

    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.show()
```

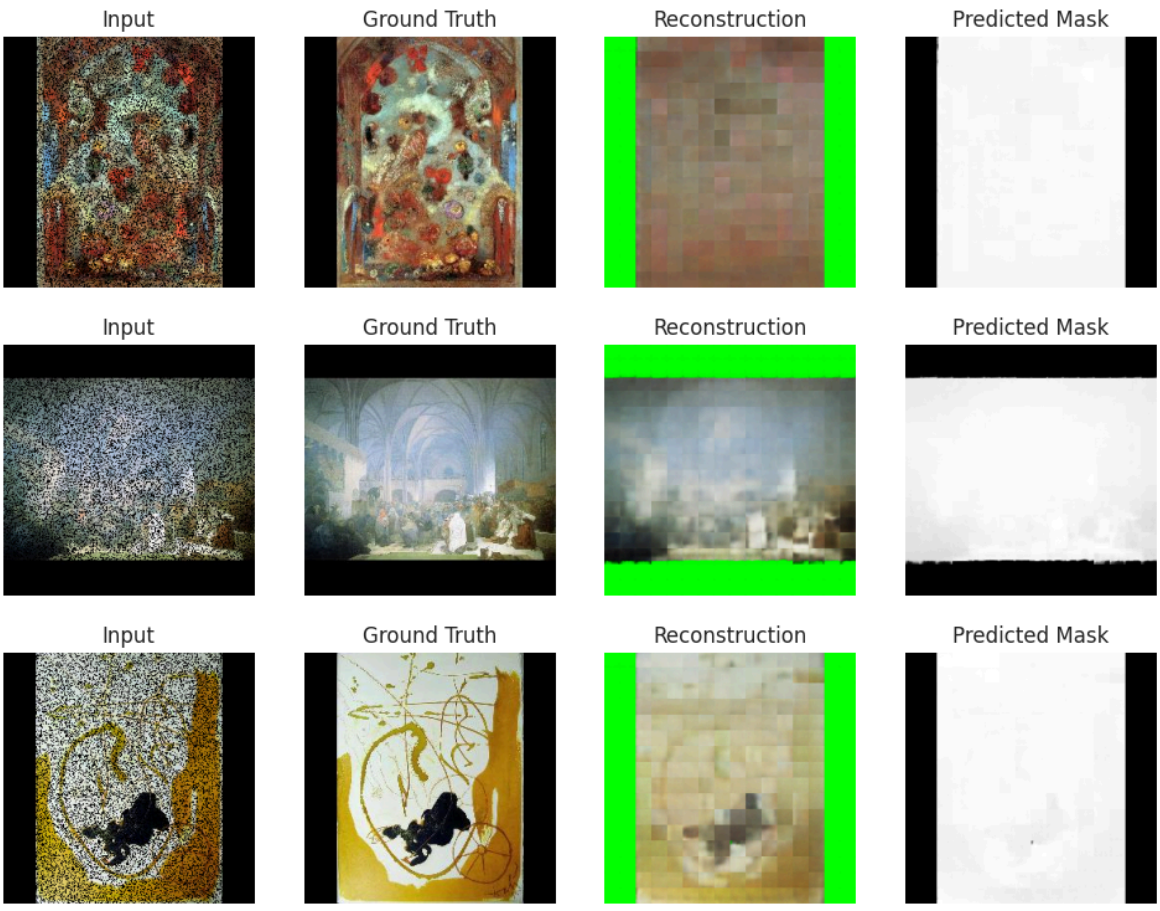
In [22]:
```
plot_training_log(log_path="training_log.csv")
```



In [23]:
```
import matplotlib.pyplot as plt
import torch.nn.functional as F

def visualize_predictions(model, val_loader, device, num_samples=3, patch
    model.eval()
    ph, pw = patch_size

    samples_shown = 0
    with torch.no_grad():
        for batch in val_loader:
            x = batch["input"].to(device)
            y = batch["target"].to(device)
            mask = batch["mask"].to(device)
            B, C, H, W = x.shape

            # Compute HW tensor
            hw = torch.tensor([[H, W]]*B, dtype=torch.float32, device=dev

            # Split into patches
            patches = [x[:, :, i:i+ph, j:j+pw]
                       for i in range(0, H, ph)
                       for j in range(0, W, pw)]

            # Model forward
```

```
            recon_patches, pred_masks = model((patches, hw))

            # Reassemble full images
            recon = torch.zeros_like(x)
            pmask = torch.zeros_like(mask)
            idx = 0
            for i in range(0, H, ph):
                for j in range(0, W, pw):
                    recon[:, :, i:i+ph, j:j+pw] = recon_patches[idx]
                    pmask[:, :, i:i+ph, j:j+pw] = pred_masks[idx]
                    idx += 1

            # Plot a few samples
            for i in range(min(B, num_samples)):
                fig, axs = plt.subplots(1, 4, figsize=(12, 3))
                axs[0].imshow(x[i].permute(1, 2, 0).cpu().numpy())
                axs[0].set_title("Input")
                axs[1].imshow(y[i].permute(1, 2, 0).cpu().numpy())
                axs[1].set_title("Ground Truth")
                axs[2].imshow(recon[i].permute(1, 2, 0).cpu().numpy())
                axs[2].set_title("Reconstruction")
                axs[3].imshow(pmask[i].permute(1, 2, 0).cpu().numpy(), cm
                axs[3].set_title("Predicted Mask")
                for ax in axs: ax.axis("off")
                plt.show()

                samples_shown += 1
                if samples_shown >= num_samples:
                    return
```

In [24]:
```
visualize_predictions(model, val_loader, device, num_samples=10, patch_si
```




Input
Ground Truth
Reconstruction
Predicted Mask
Input
Ground Truth
Reconstruction
Predicted Mask
Input
Ground Truth
Reconstruction
Predicted Mask
Input
Ground Truth
Reconstruction
Predicted Mask
Input
Ground Truth
Reconstruction
Predicted Mask
Input
Ground Truth
Reconstruction
Predicted Mask

In [25]:
```
# from tensorflow.keras import layers, models
# from tensorflow.keras.optimizers import AdamW
# from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint,
# from tensorflow.keras.optimizers import Adam, AdamW
```

In [26]:
```
# from tensorflow.keras import mixed_precision
# policy = mixed_precision.Policy('float32')
# mixed_precision.set_global_policy(policy)
```

In [27]:
```
# import tensorflow as tf
# from tensorflow.keras import layers, Model, callbacks, metrics

# # -------------------------
# # Variational Transformer Encoder (expects embedded inputs)
# # -------------------------
# class VariationalTransformerEncoder(Model):
#     def __init__(self, latent_dim=64, context_n=5, ff_dim=128, num_head
#         super().__init__(**kwargs)
#         self.latent_dim = latent_dim
#         self.context_n = context_n
#         self.ff_dim = ff_dim
#         self.pos_emb = layers.Embedding(input_dim=context_n + 1, output
#         self.attn1 = layers.MultiHeadAttention(num_heads=num_heads, key
#         self.attn2 = layers.MultiHeadAttention(num_heads=num_heads, key
#         self.ffn = tf.keras.Sequential([
#             layers.Dense(ff_dim * 2, activation='relu'),
#             layers.Dense(ff_dim),
#         ])
#         self.norm1 = layers.LayerNormalization()
#         self.norm2 = layers.LayerNormalization()
#         self.norm3 = layers.LayerNormalization()
#         self.z_mean = layers.Dense(latent_dim)
#         self.z_log_var = layers.Dense(latent_dim)

#     def call(self, seq_embedded, training=False):
#         x = seq_embedded
#         seq_len = tf.shape(x)[1]
#         positions = tf.range(seq_len)
#         x += self.pos_emb(positions)
#         a1 = self.attn1(x, x)
#         x = self.norm1(x + a1)
#         a2 = self.attn2(x, x)
#         x = self.norm2(x + a2)
#         f = self.ffn(x)
#         x = self.norm3(x + f)
#         x_pool = tf.reduce_mean(x, axis=1)
#         z_mean = self.z_mean(x_pool)
#         z_log_var = tf.clip_by_value(self.z_log_var(x_pool), -4.0, 4.0)
#         eps = tf.random.normal(tf.shape(z_mean), dtype=z_mean.dtype)
```

```
#         z = z_mean + tf.exp(0.5 * z_log_var) * eps
#         return z, z_mean, z_log_var

# # ------------------------
# # Patch Decoder
# # ------------------------
# class PatchDecoder(Model):
#     def __init__(self, patch_shape=(50,50,3), **kwargs):
#         super().__init__(**kwargs)
#         H, W, C = patch_shape
#         self.H, self.W, self.C = H, W, C
#         h8, w8 = (H+7)//8, (W+7)//8
#         self.dense = layers.Dense(h8*w8*128, activation='relu')
#         self.reshape = layers.Reshape((h8, w8, 128))
#         self.deconv1 = layers.Conv2DTranspose(128, 3, strides=2, paddin
#         self.deconv2 = layers.Conv2DTranspose(64, 3, strides=2, padding
#         self.deconv3 = layers.Conv2DTranspose(32, 3, strides=2, padding
#         # Final conv to get C channels
#         self.output_conv = layers.Conv2D(C, 3, padding='same', activati

#     def call(self, z, training=False):
#         x = self.dense(z)
#         x = self.reshape(x)
#         x = self.deconv1(x)
#         x = self.deconv2(x)
#         x = self.deconv3(x)
#         # Now map to original channels and crop
#         x = self.output_conv(x)
#         return x[:, :self.H, :self.W, :]

# # ------------------------
# # Chunked VAE with Embedded Context
# # ------------------------
# class ChunkedVAE(Model):
#     def __init__(self, encoder, decoder, h_splits, w_splits, lambda_reg
#         super().__init__()
#         self.encoder = encoder
#         self.decoder = decoder
#         self.h_splits = h_splits
#         self.w_splits = w_splits
#         self.lambda_reg = lambda_reg
#         self.context_n = context_n
#         self.patch_embed = layers.Dense(ff_dim, activation='relu')
#         self.kl_weight = tf.Variable(0.0, trainable=False)
#         self.total_loss_tracker = metrics.Mean('total_loss')
#         self.recon_loss_tracker = metrics.Mean('recon_loss')
#         self.kl_loss_tracker = metrics.Mean('kl_loss')

#     @property
#     def metrics(self):
#         return [self.total_loss_tracker, self.recon_loss_tracker, self.

#     def _extract_and_embed(self, inputs):
#         patches = tf.image.extract_patches(
#             inputs,
#             sizes=[1,self.decoder.H,self.decoder.W,1],
#             strides=[1,self.decoder.H,self.decoder.W,1],
#             rates=[1,1,1,1],
#             padding='VALID'
#         )
```

```
#         B = tf.shape(inputs)[0]
#         P = self.h_splits * self.w_splits
#         feats = self.decoder.H * self.decoder.W * self.decoder.C
#         flat = tf.reshape(patches, [B, P, feats])
#         embedded = self.patch_embed(flat)
#         return flat, embedded

#     def _gather_context(self, embedded):
#         B = tf.shape(embedded)[0]
#         P = self.h_splits * self.w_splits
#         idx = tf.range(P)
#         # Build sliding window indices
#         def ctx(i):
#             start = tf.maximum(0, i-self.context_n)
#             pad = self.context_n - (i-start)
#             inds = idx[start:i]
#             return tf.concat([tf.zeros(pad, dtype=tf.int32), inds], axi
#         ctx_idx = tf.stack([ctx(i) for i in range(P)], axis=0)
#         # Expand to batch
#         batch_idx = tf.range(B)[:,None,None]
#         batch_idx = tf.tile(batch_idx, [1,P,self.context_n])
#         ctx_idx = tf.tile(ctx_idx[None], [B,1,1])
#         gather_idx = tf.stack([batch_idx, ctx_idx], axis=-1)
#         return tf.gather_nd(embedded, gather_idx)

#     def call(self, inputs, training=False):
#         B = tf.shape(inputs)[0]
#         C = tf.shape(inputs)[-1]
#         flat, embedded = self._extract_and_embed(inputs)
#         context = self._gather_context(embedded)
#         # Reshape current embeddings
#         current = tf.reshape(embedded, [B, self.h_splits*self.w_splits,
#         seq = tf.concat([context, current], axis=2)
#         seq_flat = tf.reshape(seq, [-1, self.context_n+1, tf.shape(embe
#         z, zm, zl = self.encoder(seq_flat, training=training)
#         recon_p = self.decoder(z, training=training)
#         # Stitch patches back to image
#         recon = tf.reshape(recon_p, [B, self.h_splits, self.w_splits, s
#         recon = tf.transpose(recon, [0,1,3,2,4,5])
#         return tf.reshape(recon, [B, self.h_splits*self.decoder.H, self

#     def train_step(self, data):
#         x = data[0] if isinstance(data, tuple) else data
#         with tf.GradientTape() as tape:
#             flat, embedded = self._extract_and_embed(x)
#             context = self._gather_context(embedded)
#             B = tf.shape(embedded)[0]
#             P = self.h_splits * self.w_splits
#             current = tf.reshape(embedded, [B, P, 1, tf.shape(embedded)
#             seq = tf.concat([context, current], axis=2)
#             seq_flat = tf.reshape(seq, [-1, self.context_n+1, tf.shape(
#             z, zm, zl = self.encoder(seq_flat, training=True)
#             preds = self.decoder(z, training=True)
#             orig = tf.reshape(flat, [-1, self.decoder.H, self.decoder.W
#             orig_cast = tf.cast(orig, preds.dtype)
#             recon_loss = tf.reduce_mean(tf.square(orig_cast - preds))
#             kl_loss = -0.5 * tf.reduce_mean(1 + zl - tf.square(zm) - tf
#             loss = recon_loss + self.kl_weight * kl_loss
#         grads = tape.gradient(loss, self.trainable_weights)
#         self.optimizer.apply_gradients(zip(grads, self.trainable_weight
```

```
#         self.total_loss_tracker.update_state(loss)
#         self.recon_loss_tracker.update_state(recon_loss)
#         self.kl_loss_tracker.update_state(kl_loss)
#         return {m.name: m.result() for m in self.metrics}

#     def test_step(self, data):
#         x = data[0] if isinstance(data, tuple) else data
#         # Mirror train_step without gradients
#         flat, embedded = self._extract_and_embed(x)
#         context = self._gather_context(embedded)
#         B = tf.shape(embedded)[0]
#         P = self.h_splits * self.w_splits
#         current = tf.reshape(embedded, [B, P, 1, tf.shape(embedded)[-1]
#         seq = tf.concat([context, current], axis=2)
#         seq_flat = tf.reshape(seq, [-1, self.context_n+1, tf.shape(embe
#         z, zm, zl = self.encoder(seq_flat, training=False)
#         preds = self.decoder(z, training=False)
#         orig = tf.reshape(flat, [-1, self.decoder.H, self.decoder.W, se
#         orig_cast = tf.cast(orig, preds.dtype)
#         recon_loss = tf.reduce_mean(tf.square(orig_cast - preds))
#         kl_loss = -0.5 * tf.reduce_mean(1 + zl - tf.square(zm) - tf.exp
#         loss = recon_loss + self.kl_weight * kl_loss
#         self.total_loss_tracker.update_state(loss)
#         self.recon_loss_tracker.update_state(recon_loss)
#         self.kl_loss_tracker.update_state(kl_loss)
#         return {m.name: m.result() for m in self.metrics}

# # KL Warmup Callback (unchanged)
# class KLWarmup(callbacks.Callback):
#     def __init__(self, vae, ramp_epochs=50):
#         super().__init__()
#         self.vae = vae
#         self.ramp_epochs = ramp_epochs
#     def on_epoch_end(self, epoch, logs=None):
#         w = min((epoch+1)/self.ramp_epochs, 1.0) * self.vae.lambda_reg
#         self.vae.kl_weight.assign(w)
#         tf.print(f"KL weight now {w}")

# # -------------------------
# # Example Compile & Fit
# # -------------------------
# # When using a custom train_step, compile without specifying `loss`, an
# # to let Keras call your train_step directly.
# #
# vae = ChunkedVAE(encoder, decoder, h_splits=12, w_splits=7, lambda_re
# vae.compile(
#     optimizer=tf.keras.optimizers.Adam(1e-3),
#     loss=None,  # must explicitly set loss=None when overriding train
#     metrics=[],
#     run_eagerly=True  # required for custom train_step
# )
```

In [28]:

```
# LATENT_DIM = 64
# CTX_SIZE = 10
# encoder = VariationalTransformerEncoder(latent_dim=LATENT_DIM, context_
# decoder = PatchDecoder(patch_shape=(32, 32, 3))
# autoencoder = ChunkedVAE(encoder, decoder, h_splits=8, w_splits=8, lamb

# autoencoder.compile(
#     optimizer=tf.keras.optimizers.AdamW(1e-4),
```

```
#     loss=None,    # explicitly None for custom train_step
#     metrics=[],
#     run_eagerly=True
# )
```

In [29]:
```
# EPOCHS = 5000

# # Callbacks
# ckpt_cb = ModelCheckpoint("best_autoencoder.keras", save_best_only=True
# log_cb = CSVLogger("training_log.csv", append=True)
# warmup_cb = KLWarmup(autoencoder, ramp_epochs=100)
# earlystop_cb = EarlyStopping(
#     monitor="val_total_loss",
#     patience=200,
#     restore_best_weights=True,
#     mode='min'
# )
```

In [30]:
```
# history = autoencoder.fit(
#     train_ds,
#     validation_data=val_ds,
#     epochs=EPOCHS,
#     callbacks=[log_cb, ckpt_cb, warmup_cb, earlystop_cb]
# )
```

In [31]:
```
# def show_reconstructions(model, dataset, n=5):
#     sample_batch = next(iter(dataset))
#     x_val = sample_batch[0] if isinstance(sample_batch, tuple) else sam

#     # Limit n to available samples
#     n = min(n, x_val.shape[0])

#     reconstructions = model.predict(x_val)

#     x_val = np.clip(x_val, 0, 1)
#     reconstructions = np.clip(reconstructions, 0, 1)

#     plt.figure(figsize=(12, 5))
#     for i in range(n):
#         # Original (top row)
#         plt.subplot(2, n, i+1)
#         plt.imshow(x_val[i])
#         plt.axis("off")
#         if i == 0:
#             plt.ylabel("Original")

#         # Reconstructed (bottom row)
#         plt.subplot(2, n, i+1+n)
#         plt.imshow(reconstructions[i])
#         plt.axis("off")
#         if i == 0:
#             plt.ylabel("Reconstructed")

#     plt.tight_layout()
#     plt.show()

# show_reconstructions(autoencoder, val_ds, n=5)
```

In [32]:
```
# def show_random_generations(model, n=5):
#     # Extract patch/grid info
#     ph, pw = model.decoder.H, model.decoder.W
#     hs, ws = model.h_splits, model.w_splits
#     latent_dim = model.encoder.latent_dim  # <-- use latent_dim attribu

#     plt.figure(figsize=(12, 5))

#     for i in range(n):
#         # Generate random latent codes for all patches
#         random_latents = tf.random.normal([hs * ws, latent_dim])

#         # Decode to patches
#         random_patches = model.decoder(random_latents, training=False)

#         # Stitch patches into a full image
#         B = 1
#         P = hs * ws
#         C = random_patches.shape[-1]
#         recon_reshaped = tf.reshape(random_patches, [B, hs, ws, ph, pw,
#         recon_transposed = tf.transpose(recon_reshaped, perm=[0, 1, 3,
#         random_image = tf.reshape(recon_transposed, [B, hs * ph, ws * p

#         # Plot
#         plt.subplot(1, n, i+1)
#         plt.imshow(np.clip(random_image[0].numpy(), 0, 1))
#         plt.axis("off")
#         plt.title("Random Gen")

#     plt.tight_layout()
#     plt.show()

# show_random_generations(autoencoder, n=5)
```

In [ ]:
```
```